

Dokumentacja techniczna

System aukcyjny – REST API

Przedmiot: Tworzenie usług sieciowych REST

Autor: Marcel Sabalsky

Repozytorium: <https://github.com/Sabalsky/projekt-React>

1. Opis projektu

Projekt to serwis aukcji internetowych zbudowany w architekturze REST. Użytkownik może założyć konto, wystawić przedmiot na licytację, przeglądać aukcje innych osób i składać oferty. System składa się z dwóch części: backendu udostępniającego REST API oraz frontendu w React. Frontend korzysta wyłącznie z REST API – nie łączy się bezpośrednio z bazą danych.

2. Użyte technologie

- Backend: Node.js + Express
- Baza danych: SQLite (wbudowany moduł node:sqlite, bez dodatkowych zależności)
- Autoryzacja: JWT, hasła hashowane bcryptem
- Walidacja: express-validator
- Dokumentacja API: Swagger / OpenAPI 3.0
- Logowanie: winston + morgan
- Testy: Jest + supertest
- Frontend: React + Vite + React Router
- Konteneryzacja: Docker + docker-compose

3. Architektura

Backend podzielony jest na warstwy, żeby oddzielić logikę od reszty kodu. Żądanie wędruje przez kolejne warstwy:

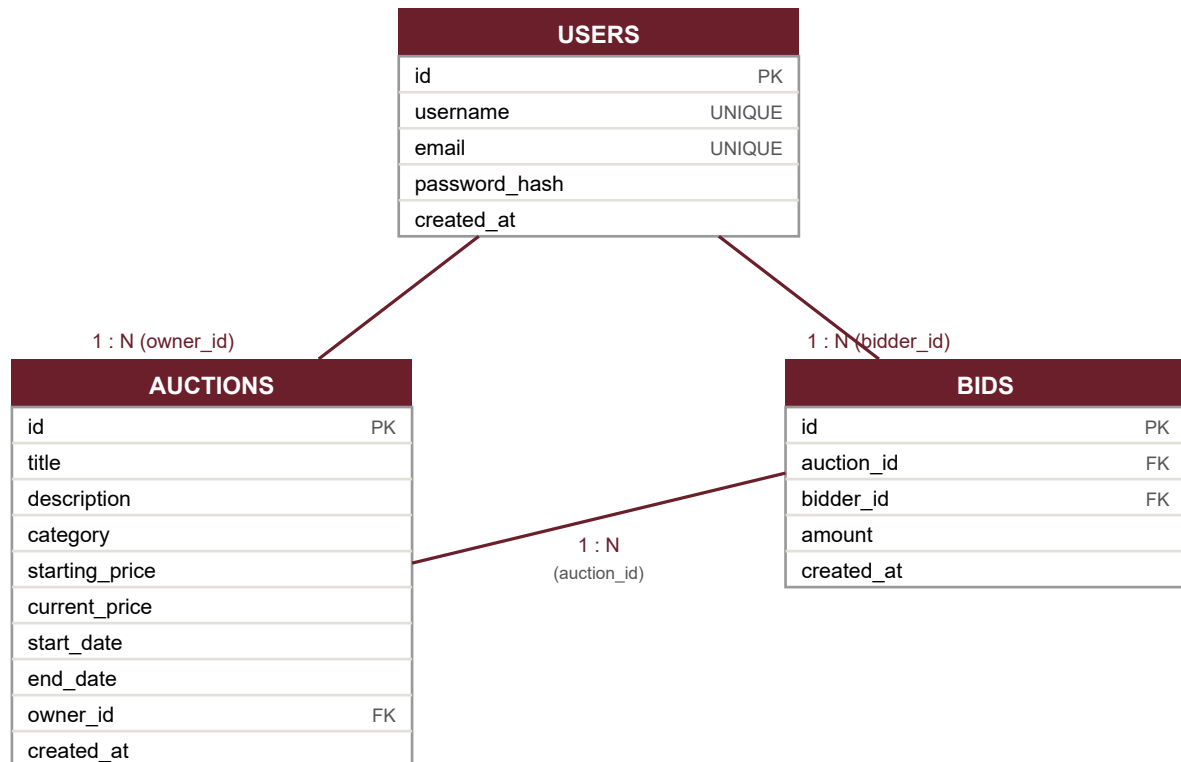
Routes → Controllers → Services → Repositories → Baza danych

Controller odbiera żądanie i zwraca odpowiedź z kodem HTTP, ale nie wie nic o SQL. Service zawiera logikę biznesową (np. sprawdzenie, czy oferta jest wyższa od aktualnej). Repository to jedyne miejsce z zapytaniami SQL. DTO zamieniają rekord z bazy na to, co zwraca API (dzięki temu hasła nigdy nie wychodzą na zewnątrz). Błędy zbiera jeden wspólny errorHandler.

```
react/
├── backend/
│   └── src/
│       ├── routes/           ścieżki (np. /auctions)
│       ├── controllers/      odbierają request, zwracają odpowiedź
│       ├── services/         logika biznesowa (reguły aukcji i licytacji)
│       ├── repositories/     zapytania SQL
│       ├── dto/              mapowanie rekordu bazy na odpowiedź API
│       ├── middleware/       JWT, walidacja, obsługa błędów
│       ├── validators/       reguły walidacji danych wejściowych
│       └── config/            baza, logger, Swagger
└── frontend/                 aplikacja React
```

4. Model danych (ERD)

Baza ma trzy tabele. Aukcja należy do użytkownika (owner_id), a oferta jest powiązana z aukcją (auction_id) i z użytkownikiem, który ją złożył (bidder_id). Ustawione jest kasowanie kaskadowe (ON DELETE CASCADE). Status aukcji nie jest trzymany w bazie – wyliczam go z dat względem aktualnego czasu.



5. Opis endpointów

Pełna, interaktywna dokumentacja jest w Swaggerze pod adresem /api-docs. Poniżej zestawienie tras.

Logowanie i użytkownicy

Metoda	Sciezka	Opis	Auth
POST	/auth/register	Rejestracja, w odpowiedzi token JWT	–
POST	/auth/login	Logowanie, w odpowiedzi token JWT	–
POST	/users	Dodanie użytkownika (to samo co rejestracja)	–
GET	/users	Lista użytkowników	–
GET	/users/:id	Pobranie jednego użytkownika	–
PUT	/users/:id	Edycja (tylko własne konto)	JWT
DELETE	/users/:id	Usunięcie (tylko własne konto)	JWT

Aukcje

Metoda	Sciezka	Opis	Auth
GET	/auctions	Lista – filtrowanie, sortowanie, paginacja	–
GET	/auctions/:id	Pobranie jednej aukcji	–

POST	/auctions	Wystawienie przedmiotu	JWT
PUT	/auctions/:id	Edycja (tylko własna aukcja)	JWT
DELETE	/auctions/:id	Usunięcie (tylko własna aukcja)	JWT

Licytacja

Metoda	Sciezka	Opis	Auth
POST	/auctions/:id/bids	Złożenie oferty	JWT
GET	/auctions/:id/bids	Historia ofert aukcji	—

Parametry GET /auctions: page, limit, category, status (active/ended/scheduled), sortBy (created_at/end_date/current_price/title), order (asc/desc).

Zwracane kody HTTP: 200, 201, 204, 400 (błąd walidacji lub złamana reguła), 401 (brak/zły token), 403 (brak uprawnień), 404 (nie ma zasobu), 409 (np. zajęty email).

6. Instrukcja uruchomienia

Wymagany Node.js w wersji co najmniej 22.

Backend:

```
cd backend
copy .env.example .env
npm install
npm run seed      # opcjonalnie: przykładowe aukcje i konta
npm start         # http://localhost:3000 (Swagger: /api-docs)
```

Frontend (drugi terminal):

```
cd frontend
npm install
npm run dev      # http://localhost:5173
```

Można też uruchomić sam backend w Dockerze: cd backend → docker compose up --build. Konta testowe po seed (hasło haslo123): alice@example.com, bob@example.com, carol@example.com.

7. Reguły licytacji

- oferta musi być wyższa niż aktualna cena, inaczej błąd 400
- nie można licytować przed startem ani po zakończeniu aukcji
- nie można licytować własnej aukcji
- po przyjęciu oferty aktualizowana jest cena aukcji i dopisywana historia ofert

8. Testy i funkcje dodatkowe

Backend ma 15 testów (Jest + supertest) uruchamianych na bazie w pamięci – sprawdzają CRUD, walidację, logowanie i wszystkie reguły licytacji (npm test).

Zrealizowane funkcje na dodatkowe punkty: autoryzacja JWT, paginacja oraz filtrowanie i sortowanie aukcji, konteneryzacja (Docker), testy jednostkowe, logowanie operacji.